MacroShock v1.0.0: Python Package for SVAR Analysis of Time Series*

Patricio Arias†

July 2026

Abstract

Structural analysis of macroeconomic shocks can provide a better explanation of random variables that realize across time like inflation rates or financial asset prices. The most popular empirical methods to identify macroeconomic shocks are Structural Vector Autorregressive (SVAR) models and, more recently, Local Projections (LP). I develop a Python library called MacroShock with a class for SVAR model creation and methods that estimate model results such as impulse response functions, forecast error variance decomposition and conditional forecasts. The package is built on top of the numerical library *Numpy*, the scientific library *SciPy* and the plotting library *Matplotlib*. Four structural identification schemes are implemented – short-run zero restrictions, long-run zero restrictions, sign restrictions and instrumental variables (proxy-SVAR) – together with normal and wild bootstrap routines to conduct inference on impulse responses, forecasts and variance decompositions. The package is validated by replicating four benchmark papers from the SVAR literature and comparing the resulting impulse responses against Cesa-Bianchi's MATLAB VAR-Toolbox v4.0.

Keywords: vector autorregression; structural shock identification; impulse response;

*Replication Python code on https://github.com/patricioariaslpz/macroshock

†Instituto Tecnológico Autónomo de México, patricio.arias@itam.mx

Contents

1 Motivation

The study of economic variables across time, often called *time series*, is a constantly evolving field in economics. The main analytic tool to study time series is econometrics, the statistical field founded in regressions. In today's world, the most popular empirical models to analyze time series are Structural Vector Autorregressive (SVAR) models and Local Projections (LP) models. Both kinds of models have advantages and disadvantages and can be used in different scenarios. Here I develop a Python package that solves generalized SVAR models.

Python is a universal and free programming language with many built-in or independent free access function libraries that make coding easier and faster. These Python libraries contain functions that run complex algorithms without having to build them ourselves. For example, the statistical libraries have functions for linear regressions such that I only have to plug in the data as inputs and in short time my output gives the estimates of the model, instead of having to write lines and lines of code for the algebra of regressions.

I develop a Python library of functions to analyze macroeconomic shocks through empirical SVAR models. The functions are built with the mathematical library *Numpy*, the scientific computing library *SciPy* and the plotting library *Matplotlib*. To my understanding, there is no existing Python library that focuses and specializes on analyzing macroeconomic shocks with empirical models such as SVAR with the four identification schemes offered here in a single, unified class.

First I introduce the fundamentals of the VAR and SVAR model, I proceed by presenting the structure of the MacroShock library and the `SVAR` class that composes it, I continue by replicating four benchmark papers that each use a different identification methodology, and finally I present some concluding remarks. Local Projections (LP) and Structural Vector Error Correction (SVEC) models are part of the package's road map and are left for future releases.

2 Introduction to Vector Autorregression

Take the 2×1 vector $\mathbf{y}_t$ with T observations defined as

$$\mathbf{y}_t = \begin{pmatrix} y_{1t} \\ y_{2t} \end{pmatrix} \quad \text{for} \quad t \in \{0, 1, 2, ..., T\}.$$

Here y_{1t} and y_{2t} are series with a time domain, that is, these variables take value for each period of time in the domain. The vector $\mathbf{y}_t$ contains $\{y_{11}, y_{12}, y_{13}, ..., y_{1T}\}$ and also contains $\{y_{21}, y_{22}, y_{23}, ..., y_{2T}\}$, which are both series that realize over time. I refer to these kind of series as *time series*. I will assume that the each time series is *covariance stationary*, that is, each y_{it} has constant mean ($\mathbb{E}[y_{it}] = \mu_i$), constant variance ($\mathbb{V}\text{ar}[y_{it}] = \sigma_i^2$) and constant auto-covariance ($\mathbb{C}\text{ov}[y_{it}, y_{it+s}] = \gamma_i(s)$).

I define a VAR model of order 1, or just a VAR(1), as

$$\mathbf{y}_t = \Phi \mathbf{y}_{t-1} + \mathbf{u}_t, \tag{1}$$

where $\mathbf{y}_t$ is the vector that contains my time series' values at period t, Φ is a 2×2 matrix with the autorregressive coefficients and $\mathbf{u}_t$ is the 2×1 vector of regression residuals. The same model I

expressed in equation (1) can be written as a system of linear equations

$$y_{1t} = \phi_{11} y_{1t-1} + \phi_{12} y_{2t-1} + u_{1t} \tag{1a}$$

$$y_{2t} = \phi_{21} y_{1t-1} + \phi_{22} y_{2t-1} + u_{2t}. \tag{1b}$$

Here ϕ_{ij} represents the coefficient of the lagged variable j in equation number i, that is the ijth element of the Φ matrix, and u_{it} represents the residual of period t in equation number i. Every VAR model can be expressed either in separate equations or in matrix notation, but I will work with matrix notation because of two main reasons. First, matrix notation is a lot more simple to write and read. Second, the code I developed works almost completely with matrix operations in order to maximize efficiency.

To estimate Φ I use OLS methodology minimizing the sum of the squared residuals, that is, I solve

$$\min_{\Phi} \sum_{t=1}^{T} \left(\mathbf{y}_t - \Phi \mathbf{y}_{t-1}\right)^2.$$

There are many ways to upgrade my simple model. For example, it is possible to add regression constants and time trend coefficients, or to increase the number of variables by increasing the dimensions of the vectors and matrices, or to extend the lagged effect by adding further period lags, or to add exogenous variables that are not contained in the endogenous vector. The general equation for the VAR(k) I will work with is stated as

$$\mathbf{y}_t = \Lambda \mathbf{z}_t + \sum_{s=1}^{k} \Phi_s \mathbf{y}_{t-s} + \Psi \mathbf{x}_t + \mathbf{u}_t.$$

Here $\mathbf{y}_t$ is a $n \times 1$ vector than contains the n endogenous time series variables, $\mathbf{z}_t$ is a 3×1 vector of deterministic variables such as a constant or a time trend, Λ is a $n \times 3$ matrix of deterministic coefficients, α_1 and α_2 are $n \times 1$ vectors for linear and squared time trend coefficients, Φ_s is the $n \times n$ matrix of coefficients for the sth lagged edogenous variable vector (for $\mathbf{y}_{t-s}$), $\mathbf{x}_t$ is a $m \times 1$ vector that contains m exogenous variables, Ψ is a $n \times m$ matrix of the exogenous variables' coefficients and u_t is the $n \times 1$ vector of regression residuals.

Therefore, my problem to solve would be

$$\min_{\{\Lambda, \Phi_1, \ldots, \Phi_k, \Psi\}} \sum_{t=k}^{T} \left(\mathbf{y}_t - \Lambda \mathbf{z}_t - \sum_{s=1}^{k} \Phi_s \mathbf{y}_{t-s} - \Psi \mathbf{x}_t \right)^2. \tag{2}$$

Notice that in problem (2) I only have $T + 1 - k$ equations, even when my sample has $T + 1$ elements. This is due to the constuction of the equations. To see this clearly, try constructing the general equation for $t = k - 1$ and notice that I do not have enough past observations to construct the kth lagged factor ($\Phi_k \mathbf{y}_{(k-1)-k} = \Phi_k \mathbf{y}_{-1}$, but there is no $t = -1$ in my sample!).

Restricted GLS Estimation

Problem (2) is solved by unrestricted OLS equation by equation whenever all coefficients are free. However, *MacroShock* also allows the user to impose linear restrictions across coefficients (for example, setting a particular lag coefficient to zero, or restricting a coefficient to be equal across

equations). To handle this in a single, numerically stable estimator I follow the restricted GLS (seemingly unrelated regression) approach of [6] (Ch. 5).

Stack the model for $t = k+1, \dots, T$ into $\mathbf{y} = (\mathbf{z}_k' \otimes \mathrm{I}_n)\boldsymbol{\beta} + \mathbf{u}$, where $\mathbf{y} = \mathrm{vec}(Y)$ stacks the $n \times (T-k)$ matrix of endogenous observations Y column by column, $\mathbf{z}_t = [\mathbf{z}_t', \mathbf{y}_{t-1}', \dots, \mathbf{y}_{t-k}', \mathbf{x}_t']'$ collects deterministic, lagged and exogenous regressors for period t, and $\boldsymbol{\beta} = \mathrm{vec}([\Lambda\ \Phi_1\ \dots\ \Phi_k\ \Psi])$ is the vector of unrestricted coefficients. Impose the linear restriction

$$\boldsymbol{\beta} = R\boldsymbol{\gamma},$$

where R is a known, full column-rank restriction matrix and $\boldsymbol{\gamma}$ is the reduced vector of free parameters (an unrestricted model simply sets $R = \mathrm{I}$). Given an estimate of the residual covariance matrix Σ_u, the feasible GLS estimator of $\boldsymbol{\gamma}$ is

$$\hat{\boldsymbol{\gamma}} = \left[R'\left(ZZ' \otimes \Sigma_u^{-1}\right)R\right]^{-1} R'\,\mathrm{vec}\left(\Sigma_u^{-1} Y Z'\right), \tag{2a}$$

where Z is the $(\dim \mathbf{z}_t) \times (T-k)$ matrix collecting $\mathbf{z}_t$ for every period. The restricted coefficient estimate is $\hat{\boldsymbol{\beta}} = R\hat{\boldsymbol{\gamma}}$, and $\hat{\Sigma}_u = (Y - \hat{B}_{\mathrm{coef}}Z)(Y - \hat{B}_{\mathrm{coef}}Z)'/(T - k - \dim \boldsymbol{\gamma})$ where $\hat{B}_{\mathrm{coef}}$ is $\hat{\boldsymbol{\beta}}$ reshaped back into an $n \times \dim \mathbf{z}_t$ matrix. Because Σ_u itself is unknown, *MacroShock* first estimates it with an unrestricted OLS pass and then plugs it into (2a); this two-step (feasible GLS) estimator is asymptotically equivalent to full maximum likelihood under Gaussian innovations. The sampling covariance of $\hat{\boldsymbol{\gamma}}$, used to build the coefficient standard errors, t-statistics and p-values reported by `SVAR.summary()`, is given by

$$\widehat{\mathbb{C}\mathrm{ov}}(\hat{\boldsymbol{\gamma}}) = \left[R'\left(ZZ' \otimes \hat{\Sigma}_u^{-1}\right)R\right]^{-1}.$$

When $R = \mathrm{I}$ this collapses to the familiar OLS-equivalent VAR estimator; when the user imposes restrictions via the `restrictions` input it delivers the constrained maximum-likelihood-type estimator described in [6].

Wold Representation

Recall the model in (1) and iterate the expression backwards, that is, writing down $\mathbf{y}_t(\mathbf{y}_{t-1}(\mathbf{y}_{t-2}(\dots)))$. In the end of the iteration backwards one would end up with an expression of the kind $\mathbf{y_t}(\mathbf{y_0})$ such as

$$\begin{aligned}
\mathbf{y}_t &= \Phi\mathbf{y}_{t-1} + \mathbf{u}_t \\
&= \Phi(\Phi\mathbf{y}_{t-2} + \mathbf{u}_{t-1}) + \mathbf{u}_t \\
&= \Phi^2\mathbf{y}_{t-2} + \Phi\mathbf{u}_{t-1} + \mathbf{u}_t \\
&\dots \\
&= \Phi^t\mathbf{y}_0 + \sum_{j=0}^{t-1} \Phi^j \mathbf{u}_{t-j}.
\end{aligned} \tag{3}$$

Notice that I can interpret equation (3) as $\mathbf{y}_t$ being composed by an initial condition discounted by the time passed and a sequence of external shocks also discounted by the distance to period t.

This representation will become important as I start working with residuals to identify shocks or to compute their impact across time on endogenous variables.

Companion Form

Recall the model in equation (1), now add one more lag to the equation and obtain

$$\mathbf{y}_t = \Phi_1 \mathbf{y}_{t-1} + \Phi_2 \mathbf{y}_{t-2} + \mathbf{u}_t. \tag{4}$$

It is possible to add the redundant equation $\mathbf{y}_{t-1} = \mathbf{y}_{t-1}$ just below equation (4) and then group elements into matrix notation such that I get a single equation, i.e. a VAR(1).

$$\begin{bmatrix} \mathbf{y}_t \\ \mathbf{y}_{t-1} \end{bmatrix} = \begin{bmatrix} \Phi_1 & \Phi_2 \\ \mathrm{I}_2 & \mathbf{0} \end{bmatrix} \begin{bmatrix} \mathbf{y}_{t-1} \\ \mathbf{y}_{t-2} \end{bmatrix} + \begin{bmatrix} \mathbf{u}_t \\ 0 \end{bmatrix}$$

$$\tilde{\mathbf{y}}_t = \tilde{\Phi} \tilde{\mathbf{y}}_{t-1} + \tilde{\mathbf{u}}_t$$

Here I_n is the $n \times n$ identity matrix. This same logic applies to VAR models of any order, that is, it is possible to reduce any VAR(k) into a VAR(1) for a finite number of lags k. To do this I only have to add the redundant equation $\mathbf{y}_{t-s} = \mathbf{y}_{t-s}$ of each lag s and group the equations into the matrix notation. For a general VAR(k) with n variables I would write

$$\begin{bmatrix} \mathbf{y}_t \\ \mathbf{y}_{t-1} \\ \dots \\ \mathbf{y}_{t-(k-2)} \\ \mathbf{y}_{t-(k-1)} \end{bmatrix} = \begin{bmatrix} \Phi_1 & \dots & \Phi_{k-2} & \Phi_{k-1} & \Phi_k \\ \mathrm{I}_n & \dots & \mathbf{0} & \mathbf{0} & \mathbf{0} \\ \dots & \dots & \dots & \dots & \dots \\ \mathbf{0} & \dots & \mathrm{I}_n & \mathbf{0} & \mathbf{0} \\ \mathbf{0} & \dots & \mathbf{0} & \mathrm{I}_n & \mathbf{0} \end{bmatrix} \begin{bmatrix} \mathbf{y}_{t-1} \\ \mathbf{y}_{t-2} \\ \dots \\ \mathbf{y}_{t-(k-1)} \\ \mathbf{y}_{t-k} \end{bmatrix} + \begin{bmatrix} \mathbf{u}_t \\ 0 \\ \dots \\ 0 \\ 0 \end{bmatrix}.$$

The pattern is clear. I denote this $nk \times nk$ companion matrix by $\mathbf{F}$ (the object stored as `SVAR.F` in the code, and used by `SVAR.stability()` to check that all its eigenvalues lie inside the unit circle). For the remainder of the paper, whenever I write Φ^s in the context of a general VAR(k) with $k > 1$ I mean the upper-left $n \times n$ block of $\mathbf{F}^s$, which is exactly the object the code computes; for the illustrative VAR(1) examples of Section 2.3 onward Φ is simply the single lag coefficient matrix, since in that case $\mathbf{F} = \Phi$.

Identification Problem

Recall the model from equation (1) and take a look at the residuals $\mathbf{u}_t$. Notice two things, first that the correlation between residuals u_{1t} and u_{2t} is non zero and second that the realization of the residual u_{1t} does not have contemporaneous effects of other variables other than y_{1t}. The first thing is tedious to prove, I will just state that, similar to the probability of randomly picking zero from the interval $(-1, 1) \in \mathbb{R}$, the probability of the covariance of two random variables being exactly zero is extremely small, virtually zero. For the second thing, notice that y_{1t} is affected by a realization in u_{1t} but y_{2t} is not. The realization of u_{1t} only affects future cross variables, such as y_{2t+1}, through its effect in y_{1t}. The narrative would be $u_{1t} \to y_{1t}(u_{1t}) \to y_{2t+1}(y_{1t}(u_{1t}))$.

I am interested in working with shocks that: 1) are uncorrelated and 2) can affect variables

contemporaneously. I impose a new upgrade to my model in equation (1) such that $\mathbf{u}_t = B\varepsilon_t$, where $\mathbf{u}_t$ is my residual vector, ε_t is the structural shock vector such that

$$\varepsilon_t \sim \mathrm{N}(0, \mathrm{I}_2), \tag{5}$$

and B is a 2×2 matrix of structural shock coefficients. Writing the model as a system of linear equations is helpful to see how the upgrade satisfies both conditions.

$$y_{1t} = \phi_{11} y_{1t-1} + \phi_{12} y_{2t-1} + b_{11}\varepsilon_{1t} + b_{12}\varepsilon_{2t} \tag{1a*}$$

$$y_{2t} = \phi_{21} y_{1t-1} + \phi_{22} y_{2t-1} + b_{21}\varepsilon_{1t} + b_{22}\varepsilon_{2t}. \tag{1b*}$$

Here, coefficient b_{ij} captures the effect that shock ε_{jt} has over variable y_{it}. Notice that from equation (5) follows directly that the structural shocks are uncorrelated between each other and that all structural shocks appear in every equation, making possible to have contemporaneous correlation between endogenous variables. However, the coefficients of B are not estimated as usual regressor coefficients since $B\varepsilon_t$ is indirectly a vector of residuals and treating it as a regressor would block my OLS methodology (no squared residuals). How can I identify B?

Let Σ_u be the 2×2 variance-covariance matrix of residuals from the reduced form VAR(2). I start from $\mathbf{u}_t = B\varepsilon_t$ and solve for B tanking variance of each side of the equation.

$$\begin{aligned}
\Sigma_u &= \mathbb{V}\mathrm{ar}(B\varepsilon_t) \\
&= BB' \\
&= \begin{bmatrix} b_{11}^2 + b_{12}^2 & b_{11}b_{21} + b_{22}b_{12} \\ b_{11}b_{21} + b_{22}b_{12} & b_{21}^2 + b_{22}^2 \end{bmatrix}
\end{aligned}$$

Apparently I have for equations, one for each matrix entry, and four unknown parameters, one for each entry of B, but the matrices are symmetrical and one equation is repeated, leaving me with only three equations and four unknown parameters. This is the identification problem. In the general case, for any VAR with n endogenous variables I have n^2 unknown parameters in B but only as many as $n(n+1)/2 < n^2$ equations ($n > 1$).

Thus, in order to identify the B matrix I must first estimate the model with reduced residuals ($\mathbf{u}_t$) and then find a B matrix such that $\Sigma_u = BB'$. But since there are more unknown parameters than equations, how do I identify B?

Identification Schemes

In order to identify B I must add restrictions to the model. The restrictions are exogenous and imposed solely by me, therefore the resulting B matrix will depend on the particular restrictions I impose.

Short-run Zero Restrictions

The easiest way to identify the B matrix is to impose short-run zero restrictions, that is, coming up with arguments to sustain why the coefficients above the diagonal are zero so I can use the lower triangular decomposition of Σ_u ($\mathrm{Chol}(\Sigma_u)$) as the structural shock coefficient matrix. Since the

lower triangular decomposition only applies to positive defined matrices, it is necessary for Σ_u to be positive defined.

As a result, my matrix $B = \text{Chol}(\Sigma_u)$ will be lower triangular implying short-run zero restrictions in a chain. In my simple model (1) my B would satisfy that $b_{12} = 0$, i.e. the contemporaneous effect of shock ε_{2t} on y_{1t} is zero. Same logic applies to the general VAR with n endogenous variables.

Long-run Zero Restrictions

Another way to identify the B matrix is to impose long-run zero restrictions. Since any VAR(k) can be reduced to a VAR(1), recall the model in equation (1). Express the model in its Wold representation for future periods to notice the impact of shocks across time.

$$\mathbf{y}_{t+s} = \Phi^t \mathbf{y}_0 + \sum_{j=0}^{t+s-1} \Phi^j B \varepsilon_{t+s-j}. \tag{6}$$

I want to calculate the aggregate impact of a shock realized in t, that is, the effect of ε_t in $\mathbf{y}_{t+s}$ with $s = \{0, 1, 2, 3, ...\}$. For each s take the elements of (6) that contain ε_t and then add across s to obtain the long-run cumulative impact of ε_t. Notice from (6) that the impact $\xi_s(\varepsilon_t)$ of ε_t on $\mathbf{y}_{t+s}$ is given by $\Phi^s B \varepsilon_t$, and therefore the long-run impact of ε_t is given by

$$\begin{aligned} \xi(\varepsilon_t) &= \sum_{s=0}^{\infty} \xi_s(\varepsilon_t) \\ &= \left[\mathrm{I}_2 - \Phi\right]^{-1} B \varepsilon_t \\ &= C \varepsilon_t. \end{aligned}$$

Now, I impose a similar restriction as in the short-term restrictions but in the long-run matrix C. That is, imposing $c_{12} = 0$, implying that in the long-run ε_{2t} has no effect over y_{1t}.

To identify B, notice that CC' is known and given by

$$\begin{aligned} CC' &= \left(\left[\mathrm{I}_2 - \Phi\right]^{-1}\right) BB' \left(\left[\mathrm{I}_2 - \Phi\right]^{-1}\right)' \\ &= D \Sigma_u D'. \end{aligned}$$

If I make restrictions such that C is lower triangular, then $\text{Chol}(D\Sigma_u D')$ is the lower triangular decomposition of $D\Sigma_u D'$ and it is equal to C, therefore

$$B = D^{-1} \text{Chol}(D \Sigma_u D').$$

The resulting B matrix will not necessarily be lower triangular and will admit contemporaneous impacts only restricted to a long-run extent. For the general VAR(k) with n endogenous variables, the long-run effects matrix D is given by the inverse of the identity matrix minus all lag coefficient matrices Φ_i [6], that is

$$D = \left(\mathrm{I}_n - \sum_{s=1}^{k} \Phi_s\right)^{-1}.$$

Sign Restrictions

Another way to identify B is via sign restrictions imposed to the entries of B. For example, if I know that ε_{2t} has a negative contemporaneous effect on y_{1t}, I should impose $b_{12} < 0$ as a restriction.

Definition: *Let* $\mathbf{I_n}$ *be the* $n \times n$ *identity matrix. A matrix* $\mathbf{X} \in \mathbb{R}^{\mathbf{n}\times\mathbf{n}}$ *is* ***orthonormal*** *if and only if*

$$\mathbf{X}\mathbf{X}' = \mathbf{I_n}.$$

Following the algorithm of [8], let X be a random 2×2 orthonormal matrix drawn from the Haar (uniform) distribution over the orthogonal group – obtained in practice by taking the Q factor of the QR decomposition of a matrix of independent standard normal draws and normalizing the sign of its diagonal – Σ_u the 2×2 variance-covariance matrix of the model presented in equation (1) and $C = \text{Chol}(\Sigma_u)$ the 2×2 lower triangular decomposition of Σ_u, then the 2×2 matrix $\tilde{B} = CX$ is a B candidate.

$$\begin{aligned}\tilde{B}\tilde{B}' &= CXX'C' \\ &= \Sigma_u.\end{aligned}$$

One can obtain an infinite amount of orthonormal X matrices such that $\tilde{B}\tilde{B}' = \Sigma_u$, therefore I can estimate more than one structural matrix B. This methodology identifies a set $\mathbb{B}$ of potential B matrices.

By imposing sign restrictions I restrict the set $\mathbb{B}$ to only contain those $\tilde{B}$ candidates that satisfy them. If I have reasons to believe that the contemporaneous impact of ε_{2t} on y_{2t} is positive, then I could restrict matrices $\tilde{B}$ such that $b_{22} > 0$. The result of this would be a set $\mathbb{B}$ of potential matrices $\tilde{B}$ such $\tilde{B}\tilde{B}' = \Sigma_u$ that and $b_{22} > 0$. One can add as many restrictions as n^2, but runtime of code will increase with each restriction.

Instrumental Variable (Proxy SVAR)

It is possible to identify columns of the B matrix with external instruments, that is, to use a proxy of a structural shock to identify its structural coefficient in B (proxy-SVAR, [10], [7]). Recall the model in (1) and assume there is an instrument z_t such that

$$\begin{aligned}\mathbb{E}[\varepsilon_{1t} z_t] &= c \\ \mathbb{E}[\varepsilon_{2t} z_t] &= 0,\end{aligned}$$

that is, an instrument that is correlates with my shock of interest ε_{1t} and uncorrelated with other shocks.

Next thing is to regress u_{1t} with z_t in what is often called first stage regression ($u_{1t} = \beta z_t + \xi_t$). As a result of this regression I will get

$$\beta = \frac{b_{11} c}{\mathbb{E}[z_t^2]}, \tag{7}$$

see appendix 1 for procedure. I can not really identify b_{11} by solving for it in equation (7) because

c is unknown. Now take $\hat{u}_{1t} = \beta z_t$ and regress

$$u_{2t} = \gamma \hat{u}_{1t} + \eta_t$$

to identify b_{21} in what is often called the second stage. The estimated parameter of the second stage regression is given by

$$\gamma = \frac{b_{21}}{b_{11}},$$

see appendix 2 for procedure.

It is usual to normalize $b_{11} = 1$ so that for a VAR with n endogenous variables I have

$$b_{i1} = \gamma_{i1}$$

where γ_{i1} is the OLS estimator of the coefficient in regression

$$u_{it} = \gamma_{i1}\hat{u}_{1t} + \eta_{it}, \qquad i \neq 1.$$

In practice usually only one column is identified with an instrument.

Instrumental Variable with Sign Restrictions

I have seen that it is possible to identify columns with instruments, but what if some columns are left unidentified? One can mix the instrument approach with the sign restrictions to fill the gap.

Suppose that, as seen before, I identify the first column of the B matrix from a VAR with n endogenous variables using the recursive regression procedure above, obtaining an unnormalized column $\mathbf{b}_1$ (with $b_{11} = 1$ by the normalization convention adopted in that procedure). Let $C = \text{Chol}(\Sigma_u)$ be the lower triangular decomposition of the variance-covariance matrix of the reduced model, then I start the sign restriction complement by finding a vector $\mathbf{q}$ such that

$$C\mathbf{q} = \mathbf{b}_1.$$

Transform $\mathbf{q}$ by a scalar $\kappa = 1/\|\mathbf{q}\|$ so that $\|\kappa\mathbf{q}\| = 1$; this is required for orthonormality. Equivalently, the correctly-scaled identified column is

$$\mathbf{b}_1^{\text{norm}} = C(\kappa\mathbf{q}) = \kappa\mathbf{b}_1 = \frac{\mathbf{b}_1}{\|\mathbf{q}\|}. \tag{7a}$$

Then I take a random $n \times (n-1)$ matrix $\tilde{Q}$ such that $Q = [\ \kappa\mathbf{q} \quad \tilde{Q}\ ]$ is orthonormal, that is, all the columns in Q have norm equal to 1 and are orthogonal between each other. Then, construct $\tilde{B} = CQ$ and preserve it in the identification set $\mathbb{B}$ only if sign restrictions are satisfied. Matrix $\tilde{B}$ is a matrix B candidate since

$$\begin{aligned}\tilde{B}\tilde{B}' &= CQQ'C' \\ &= \Sigma_u.\end{aligned}$$

Impulse Response

I am particularly interested in the impact that a structural shock ε_t realization in t has in $\mathbf{y}_{t+s}$ for $s = 0, 1, 2 \ldots$, that is, I want to see the expected impact of a shock realization in the present and future of my endogenous variables. Recall equation (6) and that the impact of ε_t on $\mathbf{y}_{t+s}$ is given by

$$\xi_s(\varepsilon_t) = \Phi^s B \varepsilon_t.$$

Let $\mathbf{e}_i$ be the ith column of the $n \times n$ identity matrix, then I am interested in the impulse-response function of shock i given by

$$IR_i(s) = \Phi^s B \mathbf{e}_i. \tag{IRF}$$

Forecast Error Variance Decomposition

I define the forecast error in horizon h as that component of $\mathbf{y}_{t+h}$ that cannot be forecasted at $t+h$. The forecast error is defined as $FE_h = \mathbf{y}_{t+h} - \mathbb{E}_{t-1}[\mathbf{y}_{t+h}]$ and, after using the Wold representation in equation (6), I can see that it is given by

$$\begin{aligned} FE_h &= \Phi^t \mathbf{y}_0 + \sum_{j=0}^{t+h-1} \Phi^j B \varepsilon_{t+h-j} - \mathbb{E}_{t-1}\left[\Phi^t \mathbf{y}_0 + \sum_{j=0}^{t+h-1} \Phi^j B \varepsilon_{t+h-j}\right] \\ &= \sum_{j=0}^{h} \Phi^{h-j} B \varepsilon_{t+j} \end{aligned}$$

Take the variance of FE_h, denoting $\Theta_m = \Phi^m B$ for $m = 0, \ldots, h$, given by

$$\mathbb{V}\text{ar}(FE_h) = \sum_{m=0}^{h} \Theta_m \Theta_m' = \sum_{m=0}^{h} \left(\Phi^m\right) B B' \left(\Phi^m\right)'.$$

The ith diagonal entry of $\mathbb{V}\text{ar}(FE_h)$, $\mathbf{e}_i' \mathbb{V}\text{ar}(FE_h) \mathbf{e}_i$, is the total forecast-error variance of variable i at horizon h; the forecast error variance decomposition attributes each shock j's share of that quantity. Then the variance decomposition of FE_h for shock j in endogenous variable i is given by

$$FED_{ijh} = \frac{\sum_{m=0}^{h} \left(\mathbf{e}_i' \Theta_m \mathbf{e}_j\right)^2}{\mathbf{e}_i' \left(\sum_{m=0}^{h} \Theta_m \Theta_m'\right) \mathbf{e}_i} = \frac{\sum_{m=0}^{h} \left(\mathbf{e}_i' \Phi^m B \mathbf{e}_j\right)^2}{\sum_{k=1}^{n} \sum_{m=0}^{h} \left(\mathbf{e}_i' \Phi^m B \mathbf{e}_k\right)^2}, \tag{8}$$

see procedure in Appendix 3. By construction $\sum_{j=1}^{n} FED_{ijh} = 1$ for every variable i and horizon h, since the n shocks are mutually orthogonal and jointly account for the full forecast-error variance. Take for example the model in (1) and compute the FED for shock 1, variable 1 and $h = 0$ given by

$$FED_{110} = \frac{b_{11}^2}{b_{11}^2 + b_{12}^2},$$

then FED_{110} is the proportion of the Forecast Error with $h = 0$ of variable 1 attributed to shock 1.

Historical Decomposition

Take the Wold representation in equation (3), the shock j historical decomposition of endogenous variable i in period t is given by

$$HD_{ij}(t) = \sum_{s=0}^{t-1} \mathbf{e}_i' \Phi^s B \mathbf{e}_j \, \varepsilon_{j,t-s}.$$

Notice that one can write y_{it} as the sum of the historical decompositions across all n shocks plus an initial condition, such that

$$y_{it} = \mathbf{e}_i' \Phi^t \mathbf{y}_0 + \sum_{j=1}^{n} HD_{ij}(t).$$

This decomposition is exposed by the `SVAR.HistoricalDecomp()` method, planned for a future release (see Section 3).

3 Python

In Python, classes are used to create objects (for example a class `Chair` in a furniture store's software) with certain attributes (the chair might have different sizes, colors, materials, etc.) and methods (there might be functions to calculate prices depending on the attributes). I use a class to construct SVAR models with attributes that describe the model's characteristics and methods that build results efficiently. All the linear algebra in the package is carried out with *Numpy* [4], the Student's t distribution and constrained optimization routines are provided by *SciPy* [12], and every figure is produced with *Matplotlib* [5]. One main class is developed, the **SVAR** class, for which a significant number of methods and functions are built on the attributes of each specific model I ought to construct. Two additional classes, **SVEC** (structural vector error correction) and **LP** (local projections), are scaffolded in the package for future releases but are not yet functional. The complete documentation can be found here.

Structure

Internally, the **SVAR** class is a thin orchestration layer that delegates each numerical task to a dedicated module, so that every estimation step can be tested, replaced or reused independently of the class itself. The package is organized as follows.

- `data.py` – builds the regressand matrix Y, the regressor matrix Z (deterministic terms, lags, exogenous variables and seasonal dummies) and the restriction matrix R from the user's raw data.
- `var.py` – estimates the restricted VAR by feasible GLS (Section 2.1), and computes standard errors, t-statistics, p-values and information criteria (AIC, BIC, HQIC).
- `stats.py` – statistical utilities (Student's t CDF and two-sided p-values), built on *SciPy*.
- `identification.py` – implements the four structural identification schemes of Section 2.4: short-run zero restrictions, long-run zero restrictions, sign restrictions and instrumental variables.

- `irf.py` – computes impulse-response functions from the companion matrix F and structural matrix B, and summarizes bootstrap draws into point estimates and confidence bands.
- `fevd.py` – computes the forecast error variance decomposition of equation (8).
- `fc.py` – computes conditional point forecasts given the estimated coefficients and deterministic terms.
- `bootstrap.py` – implements normal and wild bootstrap resampling (Section 2.6) to build confidence bands for impulse responses, forecasts and variance decompositions.
- `plotting.py` – produces publication-style plots for impulse responses, forecasts and variance decompositions using *Matplotlib*.

Installation

MacroShock is distributed through the Python Package Index (PyPI) and can be installed with

```
pip install macroshock
```

The package requires Python 3.9 or later and depends on *Numpy*, *SciPy*, *pandas* and *Matplotlib*, which are installed automatically as dependencies. Once installed, the **SVAR** class is imported with

```
from macroshock import SVAR
```

SVAR

The definition of my SVAR class is given by necessary and optional inputs that will serve as the attributes of the model I am creating. The line of code necessary to create a SVAR model is

```
model_name = SVAR(input)
```

where `model_name` can be substituted with the preferable name for the specific model and `input` is the list of inputs of the form

```
input = input1, input2, ...
```

Endogenous data can be passed either as a `pandas` DataFrame (in which case `variables` selects the relevant columns) or as a raw Numpy array.

SVAR.run()

While `SVAR(input)` creates the model as the object of analysis, I need a method to solve it, identify the structural shocks and estimate results. The method `SVAR.run()` builds the regressors, estimates the reduced-form VAR by GLS (Section 2.1), identifies the structural matrix B with the scheme selected in `method`, and prints the estimation summary. It takes no required input, and the line of code should look like

```
model_name.run()
```

Optional boolean flags `showIR`, `showFC` and `showVD` additionally compute and plot impulse responses, forecasts and variance decompositions for every variable in the same call. In order to access an output

attribute one must write

```
model_name.output_name
```

SVAR.summary()

The `SVAR.summary()` method prints the coefficient table (with standard errors, t-statistics and p-values for every equation), the information criteria, and – when the model has been identified – the estimated structural matrix B. It takes no input and the line of code would be

```
model_name.summary()
```

SVAR.ImpulseResponse()

The `SVAR.ImpulseResponse()` method computes the impulse-response function of equation (IRF) for a chosen structural shock, together with bootstrap confidence bands, and optionally plots it. The line of code would be

```
model_name.ImpulseResponse(input)
```

SVAR.Forecast()

The `SVAR.Forecast()` method computes the conditional point forecast together with bootstrap confidence bands, and optionally plots the forecasted path alongside the most recent observed periods. The line of code would be

```
model_name.Forecast(input)
```

.

SVAR.VarianceDecomp()

The `SVAR.VarianceDecomp()` method computes the forecast error variance decomposition of equation (8) attributable to a chosen structural shock, together with bootstrap confidence bands (except under the `'signs'` method, for which no bootstrap is performed and only the point decomposition implied by the median accepted B matrix is reported), and optionally plots the result as a stacked area chart. The line of code would be

```
model_name.VarianceDecomp(input)
```

The input and output follow the same structure as `SVAR.ImpulseResponse()`, replacing `shock` with the target shock for the decomposition.

SVAR.stability()

The `SVAR.stability()` method checks the stability of the estimated VAR by computing the eigenvalues of the companion matrix F and verifying that all of them lie strictly inside the unit circle, a necessary condition for the Wold representation of Section 2.2 to be well defined. The line of code would be

```
model_name.stability()
```

SVAR.HistoricalDecomp()

The `SVAR.HistoricalDecomp()` method is designed to compute the historical decomposition of Section 2.5. It is scaffolded in the current release but not yet implemented, and is part of the package's road map.

4 Replications

Four papers were replicated in order to check the correct functioning of my library. In Figures 1, 2, 3 and 4 two Impulse-Response functions are presented for each variable: the teal colored curves represent those Impulse-Response functions calculated with Cesa-Bianchi's [2] MATLAB VAR-Toolbox code, and the dark red colored curves represent those Impulse-Response functions calculated with *MacroShock*.

Stock & Watson (2001)

Following [9], a VAR(4) with three endogenous variables is presented. The endogenous variables are US unemployment (unemp_t), US inflation (infl_t) and the Federal Funds Rate (ff_t). The data is monthly and the observation range spans from the January of 1965 to December of 2003.

The structural shock matrix B is identified by sign restrictions for the first column. Therefore, the model is given by the equation

$$\begin{bmatrix} \text{infl}_t \\ \text{unemp}_t \\ \text{ff}_t \end{bmatrix} = \alpha + \sum_{s=1}^{4} \Phi_s \begin{bmatrix} \text{infl}_{t-s} \\ \text{unemp}_{t-s} \\ \text{ff}_{t-s} \end{bmatrix} + B \begin{bmatrix} \varepsilon_t^1 \\ \varepsilon_t^2 \\ \varepsilon_t^3 \end{bmatrix}$$

and the restriction

$$B = \begin{bmatrix} b_{11} & 0 & 0 \\ b_{21} & b_{22} & 0 \\ b_{31} & b_{32} & b_{33} \end{bmatrix}.$$

Blanchard & Quah (1989)

Following [1], a VAR(8) with two endogenous variables is presented. The endogenous variables are US GDP growth (y_t) and US unemployment rate (u_t). The data is quarterly and the observation range spans from the first quarter of 1948 to the fourth quarter of 1987.

The structural shock matrix B is identified by imposing long-run zero restrictions, specifically by imposing that an innovation in the unemployment rate does not impact the other variables on the long-run. Therefore, the model is given by the equation

$$\begin{bmatrix} \text{y}_t \\ \text{u}_t \end{bmatrix} = \alpha + \sum_{s=1}^{8} \Phi_s \begin{bmatrix} \text{y}_{t-s} \\ \text{u}_{t-s} \end{bmatrix} + B \begin{bmatrix} \varepsilon_t^1 \\ \varepsilon_t^2 \end{bmatrix}$$

and the restriction

$$\left(\text{I}_n - \sum_{s=1}^{k} \Phi_s \right)^{-1} B = \begin{bmatrix} c_{11} & 0 \\ c_{21} & c_{22} \end{bmatrix}.$$

Uhlig (2005)

Following [11], a VAR(12) with six endogenous variables is presented. The endogenous variables are US real GDP (y_t), US inflation rate (pi_t), commodity price index (comm_t), total reserves (res_t), non borrowed reserves (nbres_t) and the Federal Funds Rate (ff_t). The data is quarterly and the observation range spans from the first quarter of 1960 to the fourth quarter of 2000.

The structural shock matrix B is identified by imposing sign restrictions on the first shock. Therefore, the model is given by the equation

$$\begin{bmatrix} \mathrm{y}_t \\ \mathrm{pi}_t \\ \mathrm{comm}_t \\ \mathrm{res}_t \\ \mathrm{nbres}_t \\ \mathrm{ff}_t \end{bmatrix} = \alpha + \sum_{s=1}^{4} \Phi_s \begin{bmatrix} \mathrm{y}_{t-s} \\ \mathrm{pi}_{t-s} \\ \mathrm{comm}_{t-s} \\ \mathrm{res}_{t-s} \\ \mathrm{nbres}_{t-s} \\ \mathrm{ff}_{t-s} \end{bmatrix} + B \begin{bmatrix} \varepsilon_t^1 \\ \varepsilon_t^2 \\ \varepsilon_t^3 \\ \varepsilon_t^4 \\ \varepsilon_t^5 \\ \varepsilon_t^6 \end{bmatrix}$$

and the restrictions $b_{21} < 0$, $b_{31} < 0$, $b_{51} < 0$ and $b_{61} < 0$.

Gertler & Karadi (2015)

Following [3], a VAR(4) with three endogenous variables is presented. The endogenous variables are US consumer price index (logcpi_t), industrial production (logip_t), 1 year T-bill ($\mathrm{gs1}_t$) and the Excess Bond Premium (ebp_t). The data is monthly and the observation range spans from July of 1979 to June of 2012.

The structural shock matrix B is identified by instrumental variable, in this case I use a proxy for exogenous high frequence variation in the Federal Funds rate. Therefore, the model is given by the equation

$$\begin{bmatrix} \mathrm{logcpi}_t \\ \mathrm{logip}_t \\ \mathrm{gs1}_t \\ \mathrm{ebp}_t \end{bmatrix} = \alpha + \sum_{s=1}^{4} \Phi_s \begin{bmatrix} \mathrm{logcpi}_{t-s} \\ \mathrm{logip}_{t-s} \\ \mathrm{gs1}_{t-s} \\ \mathrm{ebp}_{t-s} \end{bmatrix} + B \begin{bmatrix} \varepsilon_t^1 \\ \varepsilon_t^2 \\ \varepsilon_t^3 \\ \varepsilon_t^4 \end{bmatrix}$$

5 Conclusion

In this paper *MacroShock* is introduced, a Python package for the estimation and structural analysis of Vector Autoregressive models. Built exclusively on *Numpy*, *SciPy*, *pandas* and *Matplotlib*, the package's `SVAR` class unifies four widely used structural identification schemes – short-run zero restrictions, long-run zero restrictions, sign restrictions, and external-instrument (proxy-SVAR) identification – under a single, consistent interface, together with impulse-response functions, forecast error variance decomposition, conditional forecasting, and bootstrap-based inference under both normal (i.i.d.) and wild resampling.

The package was validated empirically by replicating four influential papers, each relying on a different identification strategy: [9] for sign restrictions on a single column, [1] for long-run restrictions, [11] for sign restrictions on a single shock across multiple variables, and [3] for proxy-SVAR identification. In every case, the impulse responses and forecast error variance decompositions pro-

duced by *MacroShock* closely track those obtained with Cesa-Bianchi's [2] established MATLAB VAR-Toolbox, providing evidence that the underlying estimation and identification routines are implemented correctly. Beyond reproducing existing results, I see *MacroShock*'s main contribution as one of accessibility and unification: to my knowledge it is the first Python package to offer these four identification schemes, together with a restricted GLS estimator that accommodates arbitrary linear coefficient restrictions, inside a single object-oriented interface – lowering the barrier to entry for applied macroeconomists who work primarily in Python rather than MATLAB or R.

The current release is nonetheless a first step rather than a finished product. Historical decomposition is derived in Section 2.5 but not yet implemented, and the Structural Vector Error Correction (SVEC) and Local Projections (LP) classes are scaffolded but not functional; both are natural extensions, since LP offers a complementary, less parametric route to the same impulse responses under correct specification, and SVEC generalizes the present framework to cointegrated systems. On the inference side, sign-restriction identification does not yet propagate the uncertainty of its accepted-draw set into the conditional forecast the way it does into impulse responses and variance decompositions, an asymmetry intended to be closed in a future release. I also plan to profile and optimize the impulse-response and variance-decomposition recursions for long horizons, broaden the automated test suite, and publish the full documentation through Sphinx/ReadTheDocs – all already part of the project's public roadmap.

Hopefully *MacroShock* proves useful to researchers and students working with structural macroeconometric models in Python, and I welcome community contributions as the package continues to develop.

6 References

[1] Blanchard, O. J., & Quah, D. (1989). *The dynamic effects of aggregate demand and supply disturbances. The American Economic Review*, 79(4), 655–673.

[2] Cesa-Bianchi, A. (2015). *VAR Toolbox* [Computer software]. GitHub. `https://github.com/ambropo/VAR-Toolbox`

[3] Gertler, M., & Karadi, P. (2015). *Monetary policy surprises, credit costs, and economic activity. American Economic Journal: Macroeconomics*, 7(1), 44–76. `https://doi.org/10.1257/mac.20130329`

[4] Harris, C. R., Millman, K. J., van der Walt, S. J., Gommers, R., Virtanen, P., Cournapeau, D., ... & Oliphant, T. E. (2020). *Array programming with NumPy. Nature*, 585(7825), 357–362. `https://doi.org/10.1038/s41586-020-2649-2`

[5] Hunter, J. D. (2007). *Matplotlib: A 2D graphics environment. Computing in Science & Engineering*, 9(3), 90–95. `https://doi.org/10.1109/MCSE.2007.55`

[6] Lütkepohl, H. (2005). *New introduction to multiple time series analysis*. Springer Science & Business Media. `https://doi.org/10.1007/978-3-540-27752-1`

[7] Mertens, K., & Ravn, M. O. (2013). *The dynamic effects of personal and corporate income tax changes in the United States. American Economic Review*, 103(4), 1212–1247. `https://doi.org/10.1257/aer.103.4.1212`

[8] Rubio-Ramírez, J. F., Waggoner, D. F., & Zha, T. (2010). *Structural vector autoregressions: Theory of identification and algorithms for inference. The Review of Economic Studies*, 77(2), 665–696. `https://doi.org/10.1111/j.1467-937X.2009.00578.x`

[9] Stock, J. H., & Watson, M. W. (2001). *Vector autoregressions. Journal of Economic Perspectives*, 15(4), 101–115. `https://doi.org/10.1257/jep.15.4.101`

[10] Stock, J. H., & Watson, M. W. (2012). *Disentangling the channels of the 2007–2009 recession. Brookings Papers on Economic Activity*, 2012(1), 81–156. `https://doi.org/10.1353/eca.2012.0006`

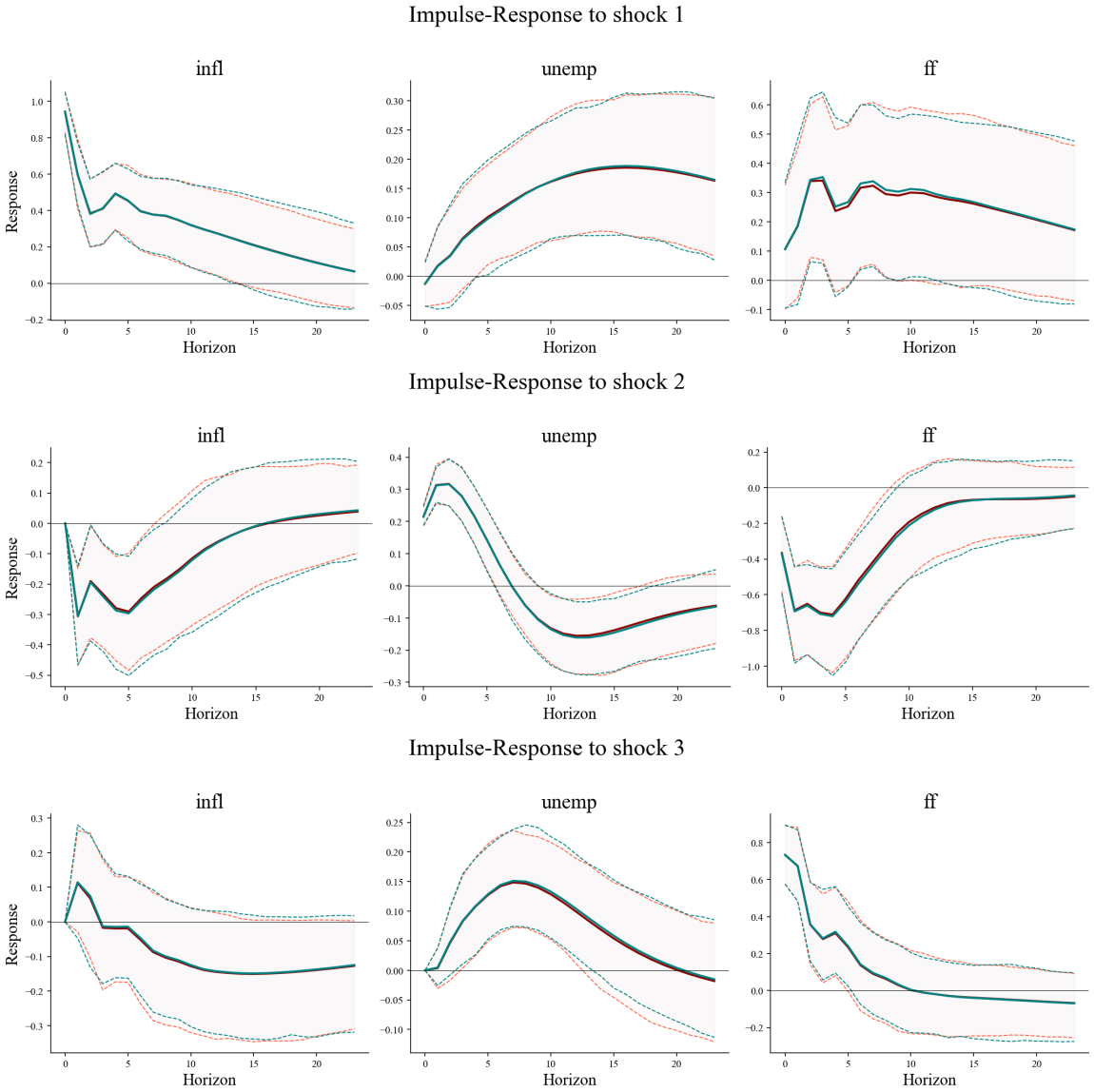
[11] Uhlig, H. (2005). *What are the effects of monetary policy on output? Results from an agnostic identification procedure. Journal of Monetary Economics*, 52(2), 381–419. `https://doi.org/10.1016/j.jmoneco.2004.05.007`

[12] Virtanen, P., Gommers, R., Oliphant, T. E., Haberland, M., Reddy, T., Cournapeau, D., ... & SciPy 1.0 Contributors (2020). *SciPy 1.0: Fundamental algorithms for scientific computing in Python. Nature Methods*, 17(3), 261–272. `https://doi.org/10.1038/s41592-019-0686-2`

7 Figures

Stock & Watson (2001)

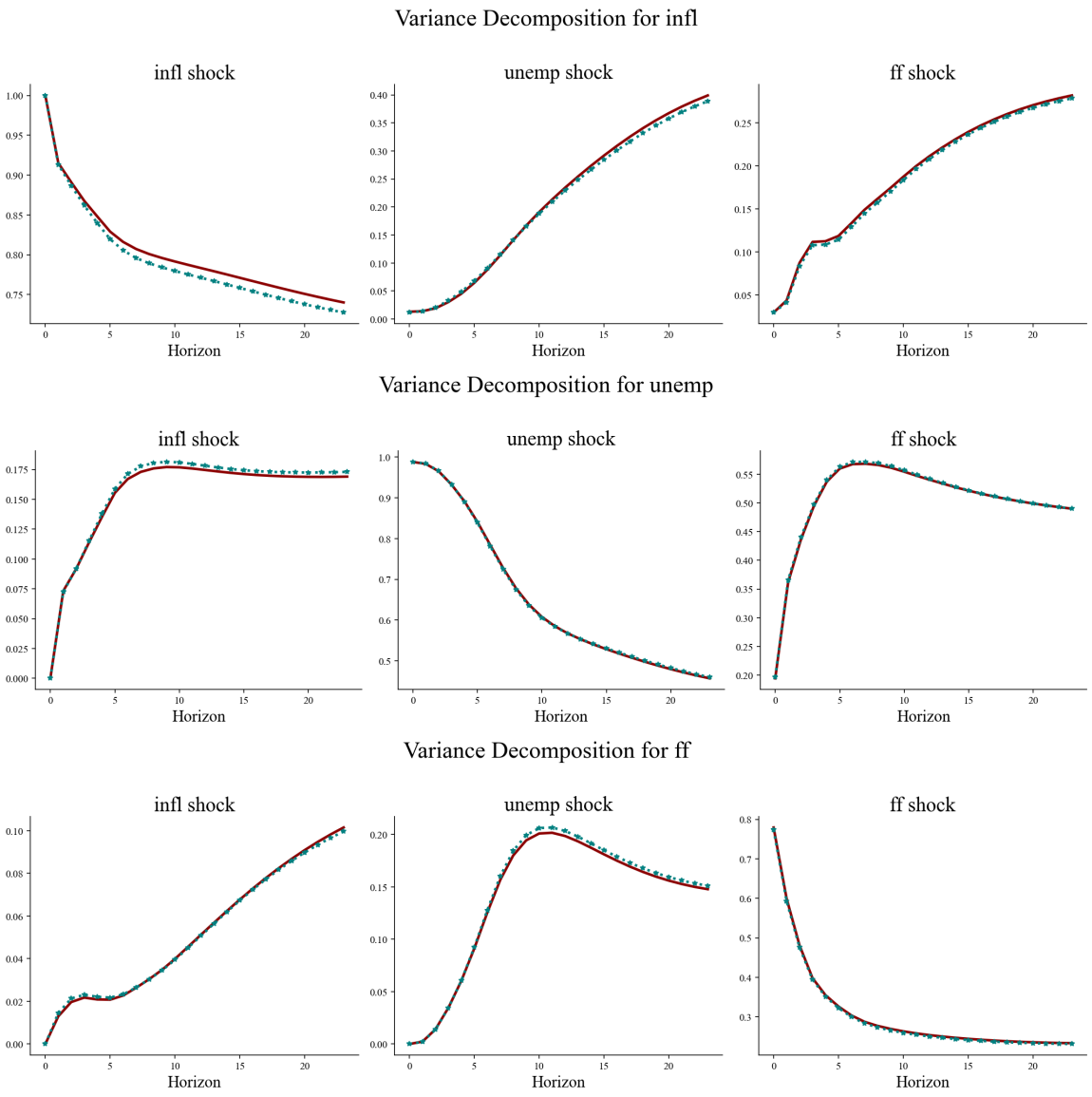
Figure 1: *Impulse response for all shocks*



Note: Red lines represent the mean IRF and the confidence bands for *macroshock* code while teal lines represent the mean IRF and the confidence bands for Cesa-Bianchi's code.

Stock & Watson (2001)

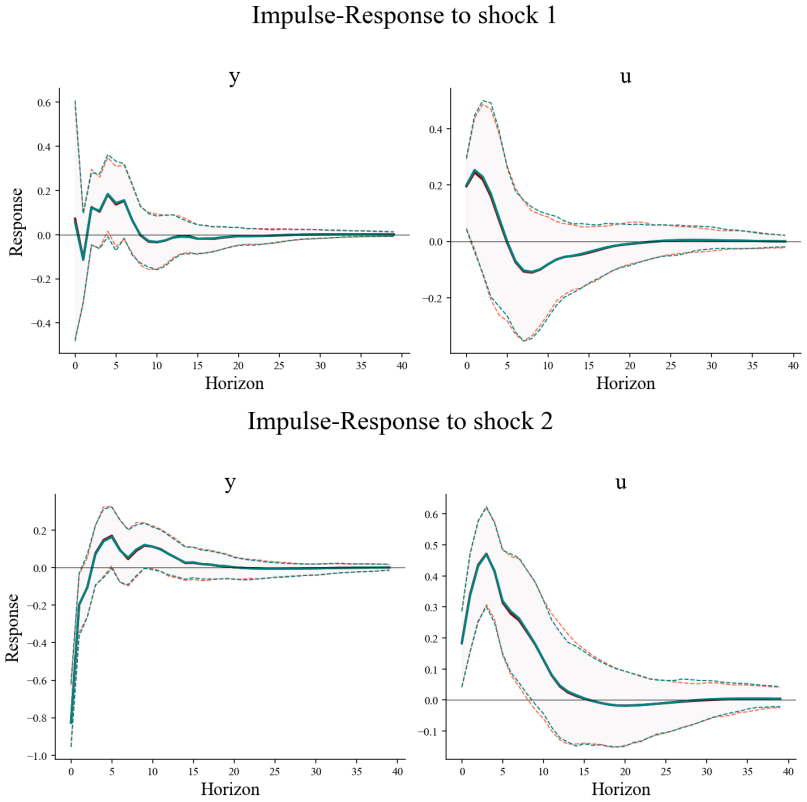
Figure 2: *FEVD for all variables*



Note: Red lines represent the mean FEVD for *macroshock* code while teal dotted lines represent the mean FEVD for Cesa-Bianchi's MATLAB code.

Blanchard & Quah (1989)

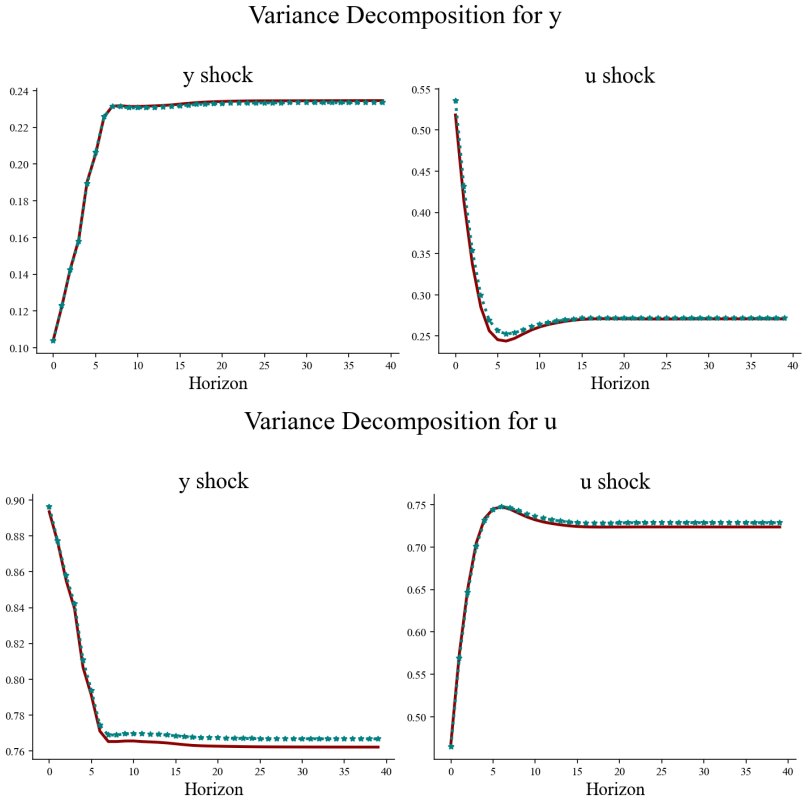
Figure 3: *Impulse response for all shocks*



Note: Red lines represent the mean IRF and the confidence bands for *macroshock* code while teal lines represent the mean IRF and the confidence bands for Cesa-Bianchi's code.

Blanchard & Quah (1989)

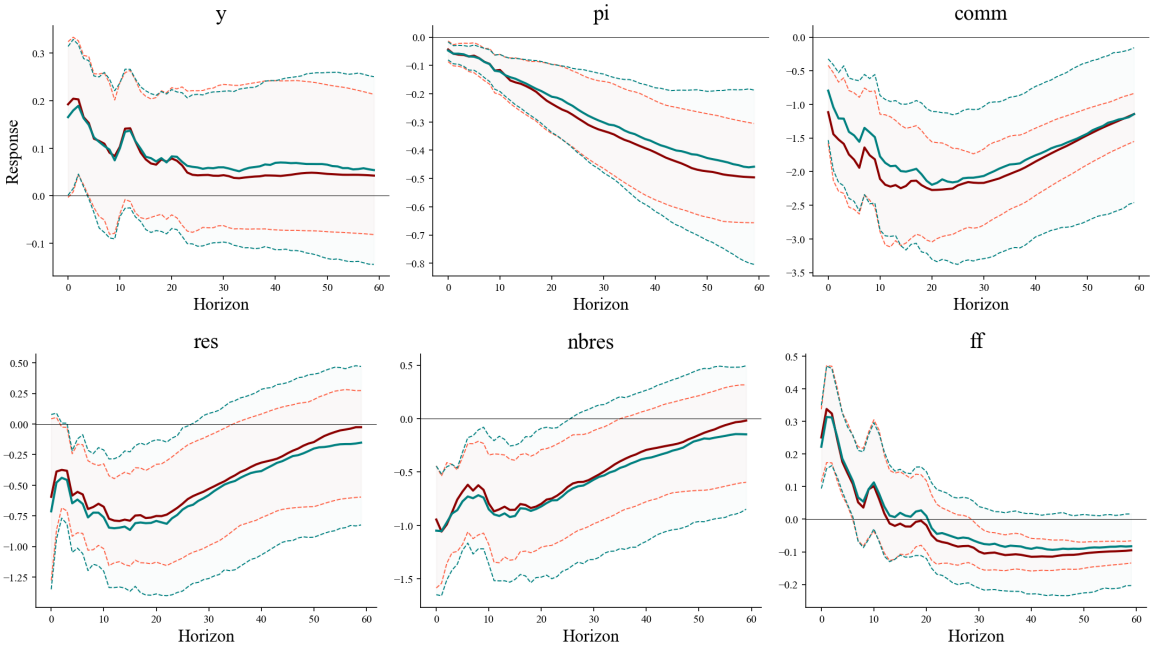
Figure 4: *FEVD for all variables*



Note: Red lines represent the mean FEVD for *macroshock* code while teal dotted lines represent the mean FEVD for Cesa-Bianchi's MATLAB code.

Uhlig (2005)

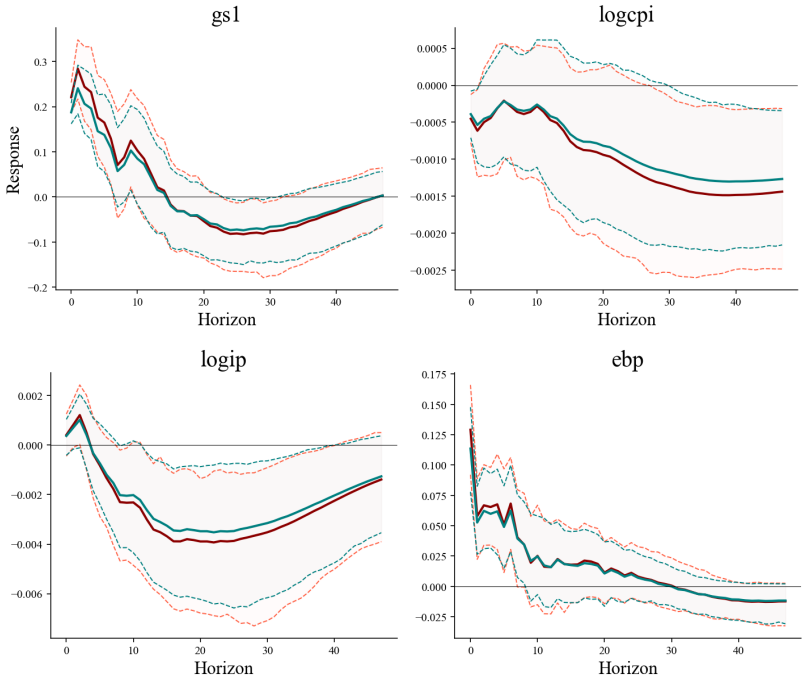
Figure 5: *Impulse response for identified shock*



Note: Red lines represent the mean IRF and the confidence bands for *macroshock* code while teal lines represent the mean IRF and the confidence bands for Cesa-Bianchi's code.

Gertler & Karadi (2015)

Figure 6: *Impulse response for identified shock*



Note: Red lines represent the mean IRF and the confidence bands for *macroshock* code while teal lines represent the mean IRF and the confidence bands for Cesa-Bianchi's code.

8 Supportive material

Appendix 1

Proposition: *Let* $\mathbf{z_t}$ *be an instrument such that*

$$\mathbb{E}[\varepsilon_{1t} z_t] = c$$
$$\mathbb{E}[\varepsilon_{2t} z_t] = 0.$$

Then the OLS coefficient of the regression $u_{1t} = \beta z_t + \xi_t$ *is given by*

$$\beta = \frac{b_{11} c}{\mathbb{E}[z_t^2]}.$$

Proof: Take the OLS optimization problem defined as

$$\min_{\beta} \mathbb{E}\left[(u_{1t} - \beta z_t)^2\right].$$

Take the FOC and solve for β.

$$[\beta]: \qquad 2\mathbb{E}\left[z_t\left(u_{1t}-\beta z_t\right)\right]=0$$

$$\beta=\frac{\mathbb{E}\left[z_t u_{1t}\right]}{\mathbb{E}\left[z_t^2\right]}.$$

Finally, substitute $u_{1t}=b_{11}\varepsilon_{1t}+b_{12}\varepsilon_{2t}$.

$$\begin{aligned}\beta&=\frac{\mathbb{E}\left[z_t(b_{11}\varepsilon_{1t}+b_{12}\varepsilon_{2t})\right]}{\mathbb{E}\left[z_t^2\right]}\\&=\frac{b_{11}c}{\mathbb{E}[z_t^2]}. \quad \blacksquare\end{aligned}$$

Appendix 2

Proposition: *Let* $\mathbf{z_t}$ *be an instrument such that*

$$\begin{aligned}\mathbb{E}[\varepsilon_{1t}z_t]&=c\\\mathbb{E}[\varepsilon_{2t}z_t]&=0.\end{aligned}$$

Then the OLS coefficient of the regression $u_{2t}=\gamma\hat{u}_{1t}+\eta_t$ *with*

$$\hat{u}_{1t}=\frac{b_{11}c}{\mathbb{E}[z_t^2]}z_t$$

is given by

$$\gamma=\frac{b_{21}}{b_{11}}.$$

Proof: Take the FOC and solve for γ.

$$[\gamma]: \qquad 2\mathbb{E}\left[\hat{u}_{1t}\left(u_{2t}-\gamma\hat{u}_{1t}\right)\right]=0$$

$$\gamma=\frac{\mathbb{E}\left[\hat{u}_{1t}u_{2t}\right]}{\mathbb{E}\left[\hat{u}_{1t}^2\right]}.$$

Finally, substitute u_{2t} and $\hat{u}_{1t}$.

$$\begin{aligned}\gamma&=\frac{\mathbb{E}\left[\frac{b_{11}c}{\mathbb{E}[z_t^2]}z_t(b_{21}\varepsilon_{1t}+b_{22}\varepsilon_{2t})\right]}{\mathbb{E}\left[\left(\frac{b_{11}c}{\mathbb{E}[z_t^2]}z_t\right)^2\right]}\\&=\frac{\frac{b_{11}b_{21}c^2}{\mathbb{E}[z_t^2]}}{\frac{b_{11}^2c^2}{\mathbb{E}[z_t^2]}}\\&=\frac{b_{21}}{b_{11}}. \quad \blacksquare\end{aligned}$$

Appendix 3

Proposition: *Let FE_h be the forecast error at horizon h such that*

$$FE_h = \sum_{m=0}^{h} \Phi^{h-m} B \varepsilon_{t+m}.$$

Then the Forecast Error Variance Decomposition of variable i attributable to shock j is given by

$$FED_{ijh} = \frac{\sum_{m=0}^{h} \left(\mathbf{e}_i' \Phi^m B \mathbf{e}_j\right)^2}{\mathbf{e}_i' \left(\sum_{m=0}^{h} \Phi^m B B' \left(\Phi^m\right)'\right) \mathbf{e}_i}.$$

Proof: Take $\Theta_m = \Phi^m B$ and take the variance of FE_h; because the shocks are independent across t (reindexing the sum with $m = h - j$) I get

$$\mathbb{V}\text{ar}(FE_h) = \sum_{m=0}^{h} \Theta_m \Theta_m'.$$

The elements in the diagonal of $\mathbb{V}\text{ar}(FE_h)$ represent the total forecast-error variance of each endogenous variable. The ith diagonal element of $\mathbb{V}\text{ar}(FE_h)$ is given by

$$\text{diag}\left(\mathbb{V}\text{ar}(FE_h)\right)_i = \mathbf{e}_i' \left(\sum_{m=0}^{h} \Theta_m \Theta_m'\right) \mathbf{e}_i = \sum_{m=0}^{h} \mathbf{e}_i' \Theta_m \Theta_m' \mathbf{e}_i = \sum_{k=1}^{n} \sum_{m=0}^{h} (\theta_m)_{ik}^2,$$

where $(\theta_m)_{ik} = \mathbf{e}_i' \Theta_m \mathbf{e}_k$ is the (i,k) entry of Θ_m. Each $(\theta_m)_{ik}$ represents the horizon-m component of variable i's forecast error explained by shock k, and $(\theta_m)_{ik}^2$ its contribution to variance. The forecast error variance decomposition of variable i attributable to shock j is the share of $\text{diag}\left(\mathbb{V}\text{ar}(FE_h)\right)_i$ explained by shock j alone, that is

$$\begin{aligned} FED_{ijh} &= \frac{\sum_{m=0}^{h} (\theta_m)_{ij}^2}{\sum_{k=1}^{n} \sum_{m=0}^{h} (\theta_m)_{ik}^2} \\ &= \frac{\sum_{m=0}^{h} \left(\mathbf{e}_i' \Phi^m B \mathbf{e}_j\right)^2}{\mathbf{e}_i' \left(\sum_{m=0}^{h} \Phi^m B B' \left(\Phi^m\right)'\right) \mathbf{e}_i}. \end{aligned}$$

■